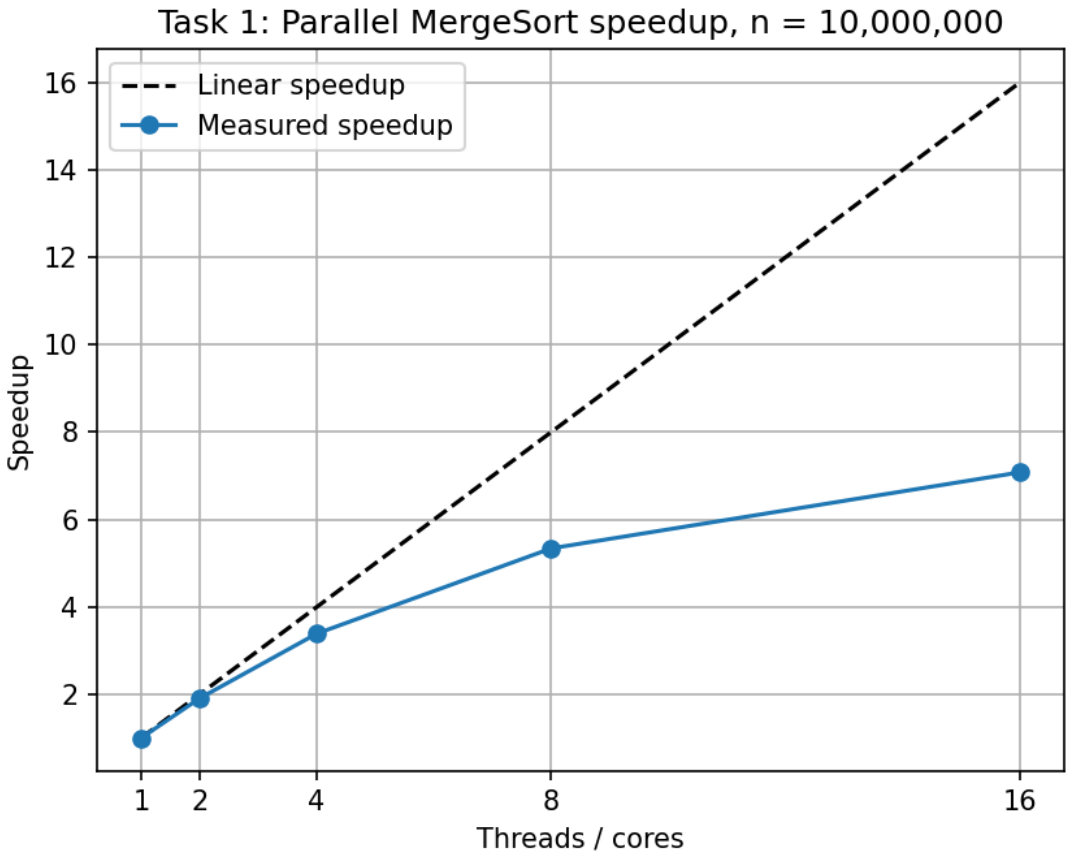
CS4475 Fall 2026
Alexa Ramirez-Salinas
aramirezsa@miners.utep.edu

Lab Report
Submitted on Oct. 29, 2026

PA3: Parallel MergeSort

Task 1. (10 pts) **Speedup analysis**

Threads	Avg sequential (s)	Avg parallel (s)	Speedup	Linear	Efficiency
1	1.5331	1.535	1	1	100%
2	1.5326	0.8004	1.91	2	96%
4	1.5325	0.4524	3.39	4	85%
8	1.5336	0.2873	5.34	8	67%
16	1.5332	0.2165	7.08	16	44%

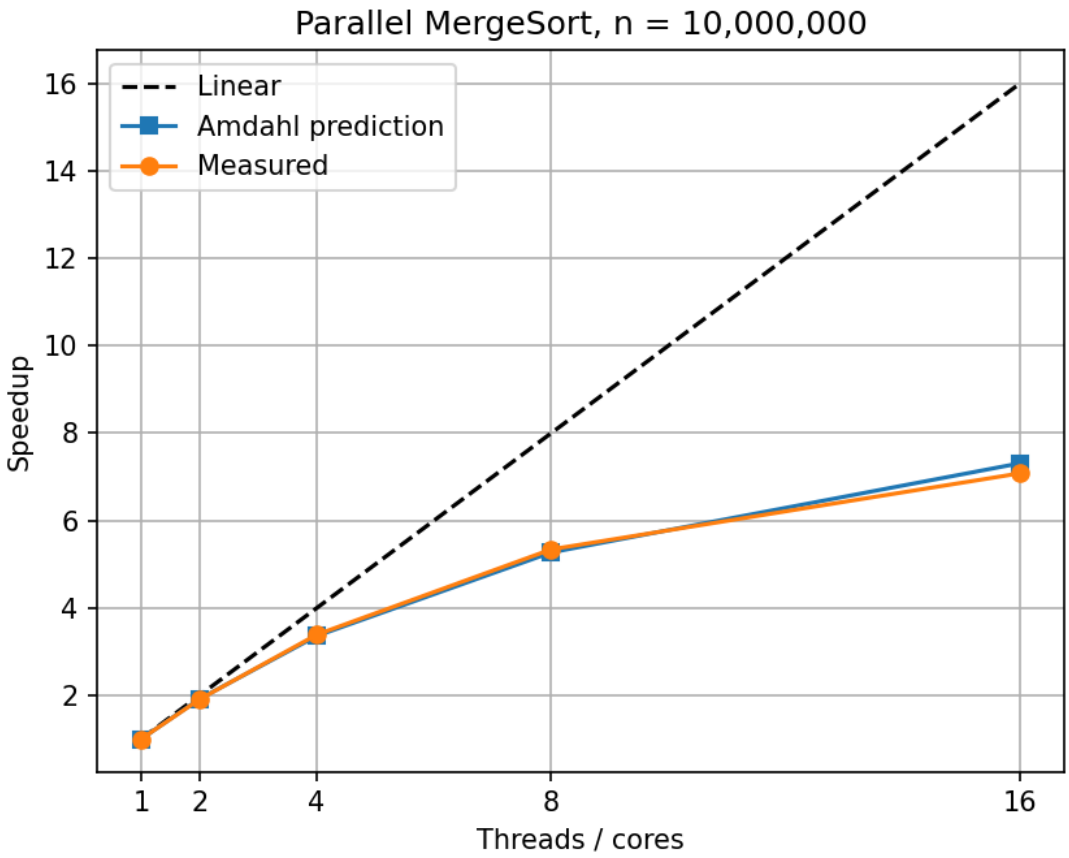


I ran the provided parallel mergesort on a 16-core Bridges-2 compute node with an input of 10,000,000 doubles. I built separate executables with THREAD_LEVEL 0 through 4 to get 1, 2, 4, 8, and 16 threads, and ran each five times. Speedup is the average sequential time divided by the average parallel time. Sequential mergesort took about 1.53 s. Speedup was close to linear

at small thread counts, 1.91 on 2 threads and 3.39 on 4 threads. It then fell away from linear, reaching 5.34 on 8 threads and 7.08 on 16 threads, which is only 44% efficiency. Going from 8 to 16 threads cut the run time by about 25% instead of 50%. The main reason is that the merge at every level of the recursion tree is sequential, so the top-level merge processes all 10 million elements on one thread no matter how many cores are available. Mergesort is also memory-bound, so threads likely compete for memory bandwidth, and the compute node is shared. An earlier set of runs (results.txt) gave about 5.6 at 16 threads and nearly identical results up to 8 threads. This shows that 16-thread timings vary between sessions on a shared node.

Task 2. (15 pts) **Amdahl's Law**

p	Top-level merge M (s)	Serial time 2M(1-1/p) (s)	Serial fraction f	Amdahl S(p)	Measured S(p)
1	n/a	0	0	1	1
2	0.0646	0.0646	0.0421	1.92	1.91
4	0.0648	0.0973	0.0635	3.36	3.39
8	0.0649	0.1136	0.0741	5.27	5.34
16	0.0648	0.1215	0.0792	7.31	7.08
∞	0.0648	0.1296	0.0845	11.8 (=1/f)	n/a

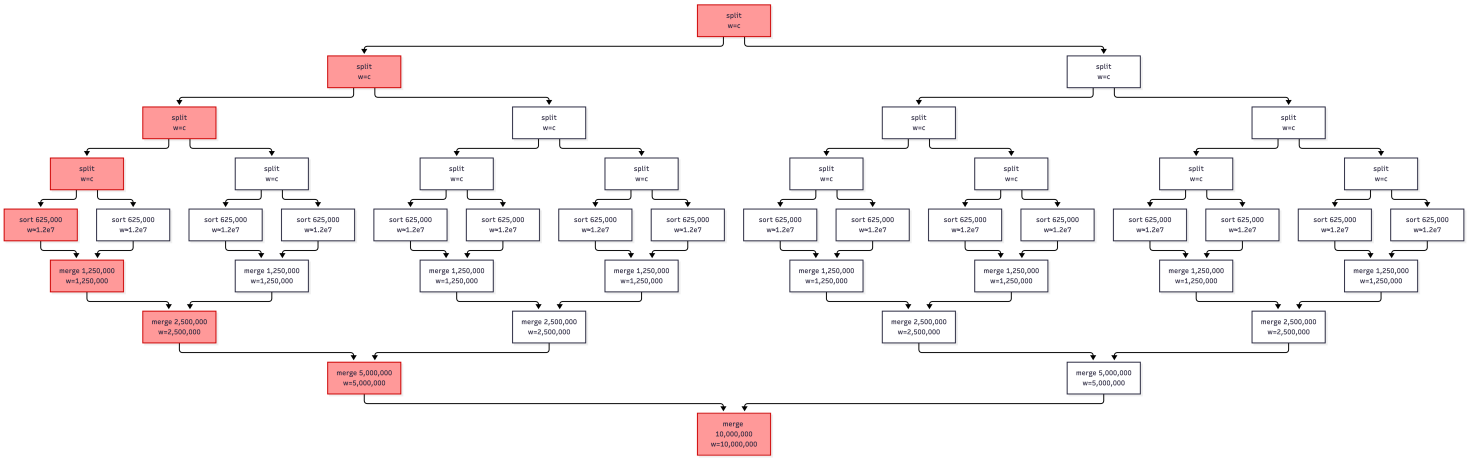


I added gettimeofday calls around the top-level merge in the parallel code and printed the elapsed time. It was about 0.065 s at every thread count, compared with about 1.53 s for the whole sequential mergesort. I could not use this value alone as the serial time. In the parallel run, the sequential merges at every level of the tree are on the critical path, because each parent must wait for its children. Those merges cover n, n/2, n/4, and so on elements, and with $\log_2(p)$ levels they sum to $2M(1 - 1/p)$. That is 0.1215 s for 16 threads. Dividing by the sequential time gives the serial fraction f: about 4.2% at 2 threads, 7.9% at 16 threads, and 8.5% in the limit. Amdahl's Law, $S(p) = 1 / (f + (1-f)/p)$, predicts 1.92, 3.36, 5.27, and 7.31 on 2, 4, 8, and 16 cores, and a maximum of about 11.8 with infinitely many cores. The measured speedups of 1.91, 3.39, 5.34, and 7.08 are within about 3% of these predictions. The small differences have plausible causes. Measured speedup is slightly above the prediction at 4 and 8 threads, probably because smaller merges lower in the tree fit better in cache than the top-level merge I used to estimate them. At 16 threads it is slightly below, likely from thread creation and join overhead, memory-bandwidth contention, and the shared node. The parallel leaf sorts are also not perfectly parallel, since they share the same memory system. The conclusion is that the sequential merge limits this algorithm, and adding cores beyond 16 would help very little.

Task 3. (15 pts) **DAG and work-span analysis (p = 16)**

Node type	Count	Size each	Work per node	Total work
Split (create 2 threads)	15	n/a	c (negligible)	15c
Leaf sequential sort	16	625,000	$(n/16)\cdot\log_2(n/16) \approx 1.20\times10^7$	$\approx 1.93\times10^8$
Merge, level 1	8	1,250,000	1.25×10^6	1.0×10^7
Merge, level 2	4	2,500,000	2.5×10^6	1.0×10^7
Merge, level 3	2	5,000,000	5.0×10^6	1.0×10^7
Merge, level 4 (top)	1	10,000,000	1.0×10^7	1.0×10^7

Quantity	Value
Work T_1	$n\cdot\log_2(n) \approx 2.33\times10^8$
Span $T\infty$ (4 splits + 1 leaf + merges n/8, n/4, n/2, n)	$\approx 1.20\times10^7 + 1.875\times10^7 \approx 3.08\times10^7$
Parallelism $T_1/T\infty$	≈ 7.5
Measured speedup at 16 threads	7.08



The DAG for 16 threads is a binary tree of 15 split nodes, each creating two child threads. Below it are 16 leaf nodes that each sequentially sort 625,000 elements, then a binary tree of 15 merge nodes, each depending on its two children. I measured work in element operations. A merge of m elements costs m, a sequential sort of m elements costs $m \cdot \log_2(m)$, and a split costs a negligible constant. Summing all nodes gives a total work of about $n \cdot \log_2(n) \approx 2.33 \times 10^8$, the same as sequential mergesort, so the parallel version does no extra work. The critical path runs down one branch through four splits, then one leaf sort of about 1.20×10^7, then the merges of sizes n/8, n/4, n/2, and n, which total 1.875×10^7. This gives a span of about 3.08×10^7. The parallelism, work divided by span, is about 7.5, so no schedule can exceed a speedup of about 7.5 on this DAG, even with unlimited processors for this thread depth. My measured 16-thread speedup of 7.08 is just below that bound. Brent's bound of $T_1/p + T\infty$ guarantees a speedup of about 5.1, so the measurement falls between the two. Converting units to time at about 6.6 ns per unit (1.53 s divided by 2.33×10^8), the span predicts about 0.20 s against a measured parallel time of 0.217 s, and the top-level merge predicts about 0.066 s against a measured 0.065 s. More than half of the span is the chain of sequential merges, so that chain is the critical path, and it explains why additional threads help so little. This agrees with the Amdahl analysis in Task 2. Replacing the sequential merge with a parallel merge would shrink the span from O(n) to polylogarithmic and raise the achievable speedup.

Task 4. (15 pts) **Reflection**

The hardest parts of this assignment were practical rather than conceptual. I had to learn the Bridges-2 workflow of logging in with ssh and then using interact to get a compute node. At first I ran programs on the shared login node, which gives unreliable timings, so I moved the runs to a compute node. I also pasted several commands while the interact job was still queued, and they got jumbled, so afterward I entered one command at a time. My first Task 2 edit put the timing code in the wrong place. It replaced the leaf-level sequential sort with a call to merge using an uninitialized midpoint. The build failed, and after a quick fix the programs crashed with free( ): invalid size and a segmentation fault. I found the bug by printing the function with `sed` and comparing it to the original. I then moved the timing code to the final merge at the bottom of the function and checked that every compile was error-free before running experiments. Copying files off the cluster failed with "subsystem request failed" and "scp: command not found" on the login node, and I solved it by using PSC's data transfer node, data.bridges2.psc.edu. I also ran a loop twice and appended to the same results file, and I had to keep each assignment on its own git branch. From this assignment I learned that speedup is limited by the sequential fraction of a program. The sequential merge caps parallel mergesort at about 7× on 16 cores and about 11.8× on infinitely many. My Amdahl prediction came within 3% of the measurement once I summed the merge levels as $2M(1 - 1/p)$ instead of using only the top-level merge. The work-span analysis gave the same conclusion from another angle: work of $n \cdot \log_2(n)$, span of about 3.1×10^7, and parallelism of about 7.5 explain why 16 cores cannot give a 16× speedup. I also learned that a better algorithm, a parallel merge, would help

more than adding threads. Good measurement needs repeated runs, identical compile flags, a compute node, and checks for correctness before trusting the numbers.

I used an AI assistant (Claude) to guide me through this assignment. It walked me through the workflow step by step: setting up the pa3 branch, building the different thread-count executables, adding timing code, running the experiments on Bridges-2, and working out the Amdahl's Law and work-span calculations. It also helped me debug problems, including the compile error and memory crash from my misplaced timing edit, and the file-transfer failures. I ran all the experiments myself on Bridges-2 and the results come from my own measurements. I checked the calculations against my output files, and I used the AI's explanations to understand the concepts (serial fraction, the $2M(1 - 1/p)$ merge-chain sum, work, span, and parallelism) rather than only copying numbers. The final write-up and interpretation of the results are my own.